

W11D01 — Layered Architecture

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 18 Mei 2026

1. Motivasi — Kenapa Ini Ada

Kode yang *works* belum tentu kode yang *maintainable*. Ketika semua logic — validasi input, query database, business rules, formatting response — berkumpul di satu file atau satu fungsi, kode itu disebut **God Object**: tahu segalanya, bertanggung jawab atas segalanya, dan hampir mustahil diubah tanpa risiko merusak bagian lain.

Layered Architecture adalah solusinya: pisahkan kode berdasarkan **tanggung jawab**, bukan berdasarkan fitur. Setiap layer hanya boleh bicara ke layer tepat di bawahnya — tidak boleh lompat.

Analogi: Restoran yang sehat punya kasir, pelayan, koki, dan gudang bahan — masing-masing punya satu peran. Kode yang sehat juga begitu. Router adalah pelayan, Service adalah koki, Repository adalah gudang.

2. Tiga Layer dan Tanggung Jawabnya

Layer	File	Tanggung Jawab	TIDAK boleh
Router	<code>routers/predict.py</code>	Terima request, validasi format via Pydantic, delegate ke service, return response	Business logic, query database, import framework ke service
Service	<code>services/score_service.py</code>	Business logic, orchestrasi, keputusan (tier, label, score)	Import FastAPI/HTTPException, query ORM langsung, tau detail HTTP
Repository	<code>repositories/user_repo.py</code>	Akses data, query database/cache/API, return objek bersih	Business logic, keputusan apapun, tau soal HTTP atau framework web

3. Aturan Emas — Yang Harus Dihafal

- Router tidak boleh ada business logic.** Satu-satunya keputusan yang boleh ada di router adalah: apakah request valid secara format (via Pydantic) dan exception mana yang di-map ke HTTP status code berapa.
- Service tidak boleh tau soal HTTP.** Import `HTTPException` di service layer adalah pelanggaran — jika besok kamu expose logic yang sama via CLI atau gRPC, service itu akan rusak.
- Repository tidak boleh ada keputusan bisnis.** `get_user_by_id()` cukup return user atau None — apakah itu 404 atau trigger email alert adalah urusan layer di atasnya.

- **Exception punya bahasa sendiri per layer.** Service lempar `ValueError` atau custom exception. Router yang menangkap dan men-translate ke `HTTPException(404)`. Seperti dapur yang bilang 'bahan habis' — pelayan yang bilang ke tamu 'maaf menu tidak tersedia'.

4. Struktur Folder Target (Week 11)

Untuk project Risk Scoring API dari Week 8, struktur yang benar setelah refactor:

```
week-11/app/  
  
    model/  
  
    ■■■■ __init__.py  
  
    ■■■■ loader.py # load_model() – infrastructure, bukan service  
  
    routers/  
  
    ■■■■ predict.py # Router layer: hanya routing + error translate  
  
    schemas/  
  
    ■■■■ predictions.py # Pydantic schema: sudah benar, tidak perlu diubah  
  
    services/  
  
    ■■■■ model_service.py # Service layer: hanya predict(), tanpa load logic  
  
    repositories/ # Akan diisi saat ada database dependency  
  
    main.py  
  
    config.py  
  
    database.py
```

5. Pola Kode yang Benar — Referensi Cepat

Service — lempar business exception, tidak import FastAPI:

```
# services/score_service.py  
  
from repositories.user_repository import UserRepository  
  
class ScoreService:  
  
    def __init__(self, user_repo: UserRepository): # DI via constructor  
        self.user_repo = user_repo  
  
    def calculate_user_score(self, user_id: int) -> dict:  
  
        user = self.user_repo.get_user_by_id(user_id)
```

```

if not user:

    raise ValueError('User tidak ditemukan') # bukan HTTPException

raw_score = user.total_purchases * 0.4 + user.account_age_days * 0.1

normalized = min(raw_score / 1000, 1.0)

tier = 'GOLD' if normalized > 0.7 else 'SILVER' if normalized > 0.4 else 'BRONZE'

return {'user_id': user_id, 'score': normalized, 'tier': tier}

```

Router — hanya routing, translate exception ke HTTP:

```

# routers/user_router.py

from fastapi import APIRouter, Depends, HTTPException

from schemas.user_schema import ScoreResponse

from services.score_service import ScoreService

from repositories.user_repository import UserRepository

router = APIRouter()

def get_score_service() -> ScoreService: # dependency function
    return ScoreService(UserRepository())

@router.get('/users/{user_id}/score', response_model=ScoreResponse)

async def get_user_score(user_id: int, service: ScoreService =
    Depends(get_score_service)):

    try:

        return service.calculate_user_score(user_id)

    except ValueError as e:

        raise HTTPException(status_code=404, detail=str(e)) # translate di sini

```

6. Pelanggaran Umum dan Cara Mengenalinya

Kode yang Bermasalah	Layer	Masalahnya	Solusi
<code>from fastapi import HTTPException</code> di service	Service	Framework leak — service tau soal HTTP	Lempar <code>ValueError</code> , router yang translate
<code>db.query(User)...</code> langsung di service	Service	Coupled ke SQLAlchemy — ganti ORM = ubah service	Buat <code>UserRepository</code> , inject ke service

Kode yang Bermasalah	Layer	Masalahnya	Solusi
<code>score = model.predict(x)[0]</code> di router	Router	Business logic bocor ke presentation layer	Pindahkan ke service layer
<code>BackgroundTasks</code> sebagai parameter service	Service	FastAPI construct masuk ke service	Tangani di router, atau pakai event callback

7. Koneksi ke Materi Sebelumnya

Layered Architecture bukan konsep yang berdiri sendiri — ia memperkuat semua yang sudah dipelajari:

- **W06D03 Preprocessing Pipeline API** — Pipeline preprocessing seharusnya ada di service layer, bukan di router. Service menerima raw features, transform, lalu predict.
- **W06D04 Structured Logging** — Dengan layer yang jelas, log menjadi informatif: 'UserRepository: user 42 not found' berbeda informasinya dengan 'Router: returning 404'. Kamu tau persis di mana masalah terjadi.
- **W09-W10 Docker & Compose** — Container tidak peduli arsitektur kode, tapi engineer yang maintain container sangat peduli. Layer yang jelas mempermudah debugging di dalam container dan memisahkan concern saat scale.
- **W05D03 Dependency Injection** — Constructor injection yang sudah dipraktikkan (ScoreService menerima UserRepository) adalah fondasi DI pattern. Minggu ini kita akan perdalam ini dengan FastAPI Depends() yang proper.

Rangkuman — Hal Paling Penting

1. Layered Architecture = pisah tanggung jawab: Router (HTTP), Service (logic), Repository (data).
2. Setiap layer hanya bicara ke layer tepat di bawahnya — tidak boleh lompat.
3. Service TIDAK BOLEH import HTTPException atau konstruk FastAPI apapun.
4. Repository adalah abstraksi cara mengambil data — service tidak peduli sumbernya dari mana.
5. Exception punya bahasa per layer: ValueError di service, HTTPException di router.
6. Struktur folder yang benar mencerminkan arsitektur — bukan hanya konvensi estetika.
7. Refactor Week 8 ke Week 11: pisahkan loader.py dari model_service.py, bersihkan FEATURE_FIELDS dari router.